

1.4 Analysing Stock Prices in R



This section builds a simple workflow:

1. Install and load packages
2. Download Microsoft and Apple price data
3. Plot each series separately, then plot them together
4. Calculate monthly and annual returns for each stock
5. Calculate CAGR for each stock
6. Import Australian house prices from FRED and compare CAGRs

Keep the goal in mind: we are learning how to do financial analysis in R, not learning R for its own sake.

Install and load packages

Note: Make sure you have installed the latest version of R: <https://cran.r-project.org/bin/windows/base/>  [\(https://cran.r-project.org/bin/windows/base/\)](https://cran.r-project.org/bin/windows/base/)

```
install.packages(c("tsibble", "fable", "feasts", "forecast", "tidyquant", "fredr"))
```

Although you will not use every package in this section, it is a good idea to install these for future use.

Explanation:

- `install.packages()` downloads packages from CRAN.
- `c(...)` combines multiple package names into one list so they can be installed at once.

Load (run each session)

```
library(tidyverse)
```

```
library(tidyquant)
```

Explanation:

- `library()` makes a package available in your current session.
-

Download Microsoft and Apple price data

```
microsoft <- tq_get("MSFT", from = "2000-01-01")
```

```
apple <- tq_get("AAPL", from = "2000-01-01")
```

Explanation:

- `tq_get()` downloads market data into a data frame.
 - "MSFT" and "AAPL" are ticker symbols.
 - `from = "2000-01-01"` sets the start date.
 - The results are stored as objects named `microsoft` and `apple`.
-

Visualise prices with ggplot2

We use the adjusted price because it accounts for stock splits and dividends and is typically preferred for return calculations.

Plot Microsoft by itself

```
ggplot(microsoft, aes(x = date, y = adjusted)) +  
  geom_line() +  
  labs(  
    title = "Microsoft Adjusted Price (MSFT)",  
    x = "Date",  
    y = "Adjusted Price (USD)"  
  ) +  
  theme_bw()
```

Explanation:

- `ggplot(microsoft, ...)` tells R which dataset to plot.
- `aes(x = date, y = adjusted)` chooses the columns used on each axis.
- `geom_line()` draws a time series line.
- `labs()` adds labels.
- `theme_bw()` uses a clean theme.

Plot Apple by itself

```
ggplot(apple, aes(x = date, y = adjusted)) +
```

```
geom_line() +  
  
labs(  
  
  title = "Apple Adjusted Price (AAPL)",  
  
  x = "Date",  
  
  y = "Adjusted Price (USD)"  
  
) +  
  
theme_bw()
```

Explanation:

Same structure as the Microsoft plot, using the Apple dataset.

Plot Microsoft and Apple together on one chart

To plot them together, we create a single combined dataset that has:

- a date column
- an adjusted price column
- a label indicating which stock each row belongs to

Add an “asset” label and combine

```
microsoft_labeled <- microsoft %>%  
  
  mutate(asset = "Microsoft (MSFT)")
```

```
apple_labeled <- apple %>%
```

```
  mutate(asset = "Apple (AAPL)")
```

```
stocks <- rbind(microsoft_labeled, apple_labeled)
```

Explanation, line by line:

- `microsoft %>% mutate(asset = "Microsoft (MSFT)")` adds a new column called `asset` that labels each row.
- `apple %>% mutate(asset = "Apple (AAPL)")` does the same for Apple.
- `rbind(...)` stacks the two tables on top of each other to form one dataset named `stocks`.

Now plot both series:

```
ggplot(stocks, aes(x = date, y = adjusted, colour = asset)) +  
  
  geom_line() +
```

```
labs(  
  title = "Adjusted Prices: Microsoft vs Apple",  
  x = "Date",  
  y = "Adjusted Price (USD)",  
  colour = "Asset"  
) +  
theme_bw()
```

Explanation:

- colour = asset tells ggplot to draw separate lines by stock.
- This produces one chart with both series.

Calculate returns

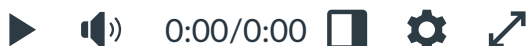
☐ Watch

Arithmetic versus log returns



Details

Transcript



Arithmetic Return

The arithmetic return measures the **percentage change in price** from one period to the next.

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} = \frac{P_t}{P_{t-1}} - 1$$

where

P_t is the price at time t (the current period) and P_{t-1} is the price at time $t-1$ (the previous period).

For example, if the current period is \$110 and the price in the previous period was \$100, then

$$R_t = \frac{110-100}{100} = 0.10 = 10\%$$

Log Return

The log return measures the continuously compounded rate of return, with

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

The log returns based on the numbers in the previous example,

$$r_t = \ln\left(\frac{110}{100}\right) = \ln(1.10) \approx 0.0953$$

Note: Log returns can be added over time.



Watch

Calculating returns



Details

Transcript



0:00/0:00



[R Script \(https://learn.usc.edu.au/courses/35359/files/3224238?wrap=1\)](https://learn.usc.edu.au/courses/35359/files/3224238?wrap=1) 
(https://learn.usc.edu.au/courses/35359/files/3224238/download?download_frd=1)

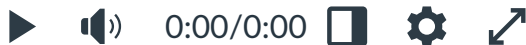
□ Watch

Log returns



Details

Transcript



[R Script \(https://learn.usc.edu.au/courses/35359/files/3224240?wrap=1\)](https://learn.usc.edu.au/courses/35359/files/3224240?wrap=1) 
(https://learn.usc.edu.au/courses/35359/files/3224240/download?download_frd=1)

Calculate monthly and annual returns in R with `tq_transmute()`

Returns can be calculated easily without doing any calculations using functions in R!

`tq_transmute()` produces a tidy return series.

Microsoft monthly returns

```
msft_monthly_returns <- microsoft %>%
```

```
  tq_transmute(
```

```
    select = adjusted,
```

```
    mutate_fun = periodReturn,
```

```
period = "monthly",  
  
type = "log" # change this to "arithmetic" if you want arithmetic returns (for analysis, log returns  
are preferable)  
  
)
```

Explanation:

- select = adjusted chooses the price series used to compute returns.
- mutate_fun = periodReturn uses a built-in return function.
- period = "monthly" means monthly returns.
- type = "arithmetic" means simple returns.

Microsoft annual returns

```
msft_annual_returns <- microsoft %>%
```

```
tq_transmute(  
  
  select = adjusted,  
  
  mutate_fun = periodReturn,  
  
  period = "annual",  
  
  type = "log"  
  
)
```

Explanation:

- Same calculation, but returns are aggregated annually.

Apple monthly returns

```
aapl_monthly_returns <- apple %>%
```

```
tq_transmute(  
  
  select = adjusted,  
  
  mutate_fun = periodReturn,  
  
  period = "monthly",  
  
  type = "log" # change to "arithmetic" as needed  
  
)
```

Apple annual returns

```
aapl_annual_returns <- apple %>%
```

```
tq_transmute(  
  

```

```
select = adjusted,  
mutate_fun = periodReturn,  
period = "annual",  
type = "log"  
)
```

Explanation:

- Apple uses the same return workflow as Microsoft.
- Only the input dataset changes.